

Cahier de refonte — Church Register

2026-09-22 · @Someone

Contexte et décisions prises

Refonte complète de Church Register (Église La Maison de la Destinée) à partir de zéro, sans reprise des données existantes. L'audit Claude Code du code actuel (Express + MongoDB + CRA) a révélé une fuite de données personnelles sur `/identify`, une XSS stockée, un double hachage des mots de passe et des fonctions inopérantes sur Render gratuit.

Décisions :

- Hébergement : Hostinger Business (PHP + MySQL, fichiers statiques). Pas de serveur Node permanent possible.
- Backend : Laravel 11 + MySQL, sur le même hébergement.
- Frontend : Vite + React + TypeScript.
- Données : nouveau schéma, base vide au lancement.
- Déploiement : GitHub Actions → branche `deploy` → Git hPanel, sans SSH ni FTP.
- À faire immédiatement en prod actuelle, avant tout : changer le mot de passe `super_admin`, le `JWT_SECRET` et le mot de passe Atlas.

Structure du dépôt

Monorepo unique, une seule CI.

```
church-register/  
  api/                Laravel 11, PHP 8.2+  
  web/                Vite + React + TypeScript  
  .github/workflows/deploy.yml  
  README.md
```

Règles : `.env` jamais commité ; `.env.example` sans valeur réelle ; Node 20 LTS et PHP 8.2 fixés dans `package.json` et `composer.json` ; ESLint, PHPStan et Pest bloquants en CI.

Schéma MySQL

Une visite par jour et un numéro unique sont garantis par des index uniques, pas par du code.

Table	Colonnes clés	Index
visitors	id, phone (E.164), full_name, whatsapp, commune, quartier, source, source_other, invited_by, consent_at, status, created_at	unique(phone), status, created_at, fulltext(full_name, commune)
visits	id, visitor_id, visit_number (1-3), visit_date (DATE), family_id, answers (JSON), idempotency_key, created_at	unique(visitor_id, visit_date), (family_id, visit_date), unique(idempotency_key)
visitor_notes	id, visitor_id, user_id, body, created_at	visitor_id
members	id, visitor_id, converted_by, converted_at	unique(visitor_id)
families	id, name, active	—
family_rotations	id, family_id, year, month	unique(year, month)
users	id, name, email, password (argon2id), role, is_active, failed_attempts, locked_until, two_factor_secret, last_login_at	unique(email), role
settings	key, value (JSON)	primary(key)
report_recipients	id, email, active	unique(email)
audit_logs	id, user_id, action, subject_type, subject_id, ip, created_at	(subject_type, subject_id), created_at
personal_access_tokens	table Sanctum standard	—

Normalisation : le numéro est converti en E.164 côté serveur avant toute lecture ou écriture. Un champ WhatsApp vide est stocké NULL, jamais +225 .

Rétention : purge automatique des visiteurs sans visite depuis 24 mois (commande planifiée), pour respecter la minimisation des données.

Parcours visiteur (API publique)

Le serveur ne renvoie jamais une donnée personnelle au public ; l'identité est portée par un jeton signé.

```
sequenceDiagram
    participant V as Visiteur
    participant A as API
    V->>A: POST /identify {country, phone}
    A-->>V: {step, session_token}
    V->>A: POST /visits {token, idempotency_key, answers}
    A-->>V: 201 créé / 200 déjà reçu
```

Route	Entrée	Sortie	Règles
POST /api/public/identify	country, phone	step (1, 2, 3 ou complete), session_token	Jeton JWT 15 min lié au numéro, usage unique par étape. Réponse identique que le numéro existe ou non.
POST /api/public/visits	session_token, idempotency_key, answers, consent	201 ou 200	Une visite par jour (index unique). Clé d'idempotence : un renvoi après coupure réseau est un succès. Le nom n'est jamais modifiable depuis le public.
GET /api/public/qr	—	verset, nom de l'église	Lu depuis <code>settings</code> , cache 60 s.

Limites de débit : 20 requêtes/min par IP, 5 identify/heure par numéro, avec `TrustProxies` configuré pour l'hébergeur.

Formulaires :

1. Visite 1 : nom, WhatsApp (optionnel), commune, quartier, source, invité par, case de consentement obligatoire avec lien vers la mention d'information.
2. Visite 2 : raisons du retour.
3. Visite 3 : motivation.

4. Au-delà : page « Parcours complet, parlez à un responsable ».

La famille d'accueil est déterminée côté serveur depuis `family_rotations` à la date de la visite.

Dashboard admin

Authentification Sanctum en cookie httpOnly sur le même domaine : pas de CORS, pas de jeton en localStorage.

- Session : cookie SameSite=Lax, CSRF Laravel, expiration après 2 h d'inactivité, déconnexion serveur réelle, changement de mot de passe qui invalide les autres sessions.
- Login : verrouillage 5 échecs / 15 min par compte, temps de réponse constant, 2FA TOTP optionnel pour super_admin.
- Rôles via Policies : lecteur (lecture), modérateur (notes, édition visiteur), super_admin (conversion, suppression, admins, paramètres QR, destinataires).
- Admins : création avec mot de passe temporaire envoyé par e-mail, activation/désactivation, un super_admin ne peut pas se rétrograder ni supprimer le dernier super_admin.

Pages : Accueil (stats avec cache 5 min), Visiteurs (filtres combinables, recherche échappée, pagination bornée 1-100), Fiche visiteur (notes), Membres, Rapports, Admins, Paramètres, QR code.

Exports : générés côté serveur en streaming, filtres actifs respectés, sans troncature. CSV avec neutralisation des cellules commençant par `= + - @`. Excel via Laravel Excel. PDF via DomPDF avec échappement HTML systématique.

Rapports mensuels : commande Artisan planifiée le 1er à 8 h, un rapport pour la famille de service uniquement, e-mail via l'API Brevo (HTML échappé), liste des visiteurs consultable dans le dashboard, sélecteur d'années dynamique.

Frontend

Deux bundles séparés : le parcours visiteur reste sous 100 Ko gzip, le dashboard ne se charge que sur `/admin`.

- Stack : Vite, React 18, TypeScript, react-router, react-hook-form + zod, TanStack Query, Tailwind.
- Parcours persistant : étape et saisie en sessionStorage, restaurés après rafraîchissement ou bouton Précédent ; historique du navigateur respecté.

- Réseau : délai d'expiration 15 s, message clair en cas d'échec, bouton désactivé pendant l'envoi, renvoi automatique avec la même clé d'idempotence.
- Téléphone : masque par pays, 10 chiffres CI acceptés avec ou sans espaces, validation zod alignée sur le serveur.
- Mobile : inputs 16 px (pas de zoom iOS), cibles tactiles 44 px minimum, formulaires natifs (touche « Aller » fonctionne), confirmation avant « Retour à l'accueil ».
- Accessibilité : `lang="fr"`, labels liés aux champs, erreurs annoncées via `aria-live`, contrastes 4,5:1 minimum, navigation clavier complète.
- Assets : logo WebP < 20 Ko, favicon, manifest et meta description au nom de l'église (aperçu WhatsApp propre).
- Textes : « Église », « 1re / 2e / 3e », « Pasteur », « Ravis ».

Sécurité transverse

Chaque point de l'audit est couvert par une règle applicable à tout le code, pas par un correctif isolé.

Risque de l'audit	Réponse dans la refonte
Fuite de données via <code>/identify</code>	Aucune donnée personnelle dans les réponses publiques ; jeton de parcours signé
XSS stockée (PDF, e-mails)	Échappement Blade par défaut, DomPDF avec <code>e()</code> , HTML e-mail via templates Blade
Double hachage, mot de passe faible	Hachage <code>argon2id</code> géré par Laravel uniquement, 12 caractères minimum
Mots de passe dans le dépôt	Compte <code>super_admin</code> créé par commande Artisan interactive, jamais de valeur en dur
Limite de débit partagée	<code>TrustProxies</code> , limites distinctes public / login / admin
Secret JWT faible	Sanctum, secrets générés (<code>php artisan key:generate</code>), distincts par environnement
Sessions non révocables	Cookies <code>httpOnly</code> , révocation en base, expiration d'inactivité
Absence de validation	Form Request obligatoire sur 100 % des routes, longueurs maximales, formats
Injection regex / formules CSV	Recherche via <code>LIKE</code> paramétré, neutralisation CSV

Risque de l'audit	Réponse dans la refonte
Scripts CDN sans SRI	Tout est bundlé, aucun script externe, CSP stricte
Erreurs verbeuses	Messages génériques au client, détails dans les logs serveur
Consentement absent	Case obligatoire, mention d'information, <code>consent_at</code> stocké

Headers : CSP stricte, HSTS, X-Frame-Options DENY, Referrer-Policy, Permissions-Policy, via middleware. Journal d'audit sur toute action admin sensible. Tests Pest : identify, idempotence, unicité par jour, permissions par rôle, verrouillage de compte.

Déploiement Hostinger sans SSH

Chaque push sur `main` met le site à jour en moins d'une minute, sans SSH ni FTP, et sans jamais partager d'identifiant.

flowchart LR

```

A[Push sur main] --> B[GitHub Actions<br/>lint, tests, build]
B --> C[Push branche deploy]
C --> D[Webhook hPanel]
D --> E[Sous-domaine à jour]

```

1. Dans hPanel : créer le sous-domaine, la base MySQL, et un cron `php artisan schedule:run` chaque minute.
 2. hPanel → Sites → sous-domaine → Avancé → Git : connecter le dépôt GitHub, branche `deploy`, dossier du sous-domaine. Copier l'URL de webhook fournie.
 3. GitHub → Settings → Webhooks : coller cette URL.
 4. Le workflow `deploy.yml` : `composer install --no-dev --optimize-autoloader`, `npm ci && npm run build`, tests, puis push du résultat dans `deploy` avec `public/` réécrit vers la racine web, `.htaccess` (HTTPS forcé, SPA, blocage des dossiers sensibles).
 5. `.env` de production rempli une seule fois via le gestionnaire de fichiers hPanel, hors dépôt. `APP_DEBUG=false`.
 6. Bascule : test complet sur le sous-domaine, mise à jour du QR code imprimé, puis arrêt de Render et Vercel.
- ☐ Sous-domaine, base et cron créés dans hPanel
 - ☐ Dépôt Git connecté et webhook posé
 - ☐ `.env` de production rempli

- ☐ Compte Brevo créé, clé API dans `.env`
- ☐ Render et Vercel arrêtés après bascule

Consigne pour Claude Code

À coller dans Claude Code, ouvert sur le dépôt, avec ce cahier.

```
Refonte complète de Church Register selon le cahier ci-joint.  
Option : Laravel 11 + MySQL sur Hostinger Business, frontend Vite + React + TypeScript.  
Repartir de zéro, sans migration des données existantes.  
Livrer phase par phase, en demandant validation avant chaque phase :  
1. Structure monorepo, migrations et modèles, seeders des familles.  
2. API publique (identify, visits, qr) avec jeton de parcours, idempotence, tests Pest.  
3. API admin (auth Sanctum, Policies, visiteurs, membres, rapports, exports, admins, paramètres).  
4. Frontend public (parcours visiteur, accessibilité, mobile).  
5. Dashboard admin.  
6. CI GitHub Actions, workflow de déploiement, .htaccess, README.  
Règles non négociables : aucune donnée personnelle dans les réponses publiques,  
validation Form Request sur chaque route, aucun secret dans le dépôt, tests bloquants.
```

Phases 1 à 3 peuvent démarrer sans accès Hostinger. La phase 6 attend la création du sous-domaine et du dépôt Git dans hPanel.